VortexTech Cybersecurity Week 3 Report

Basic Security Audit of OWASP Juice Shop

Name: Abdullah Zubair

Target Application: OWASP Juice Shop

Testing Environment: Ubuntu Virtual Machine safe lab environment with Docker container

1. Objective

The objective of this task was to perform a beginner-level security audit on a legal vulnerable-by-design practice website. For this audit, I used OWASP Juice Shop, which is intentionally vulnerable and designed for security training.

The audit focused on identifying, documenting, and manually verifying at least three vulnerability categories:

- SQL Injection
- Application Error Disclosure
- Missing Content Security Policy Header

2. Scope

This security audit was performed only against a local instance of OWASP Juice Shop running on my own machine. No real production website or third-party system was tested.

In-scope target:

OWASP Juice Shop (http://127.0.0.1:3000)

Out of scope:

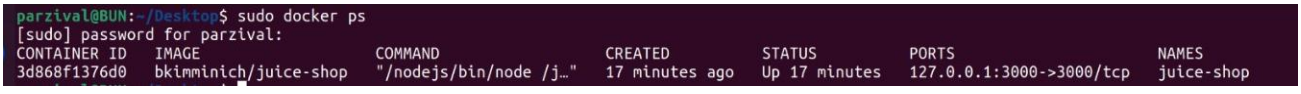
Any real public/production website
Any system without explicit permission

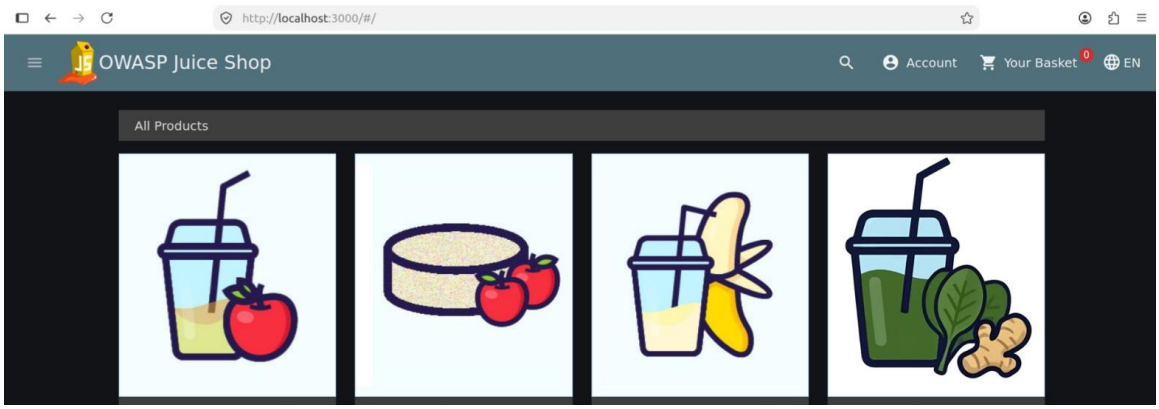
3. Environment Setup

OWASP Juice Shop was run locally using Docker. After starting the container, I verified that the container was running with:

sudo docker ps

The Docker output showed the Juice Shop container running and mapped to local port 3000. The application was then accessed in the browser at: http://localhost:3000/#/

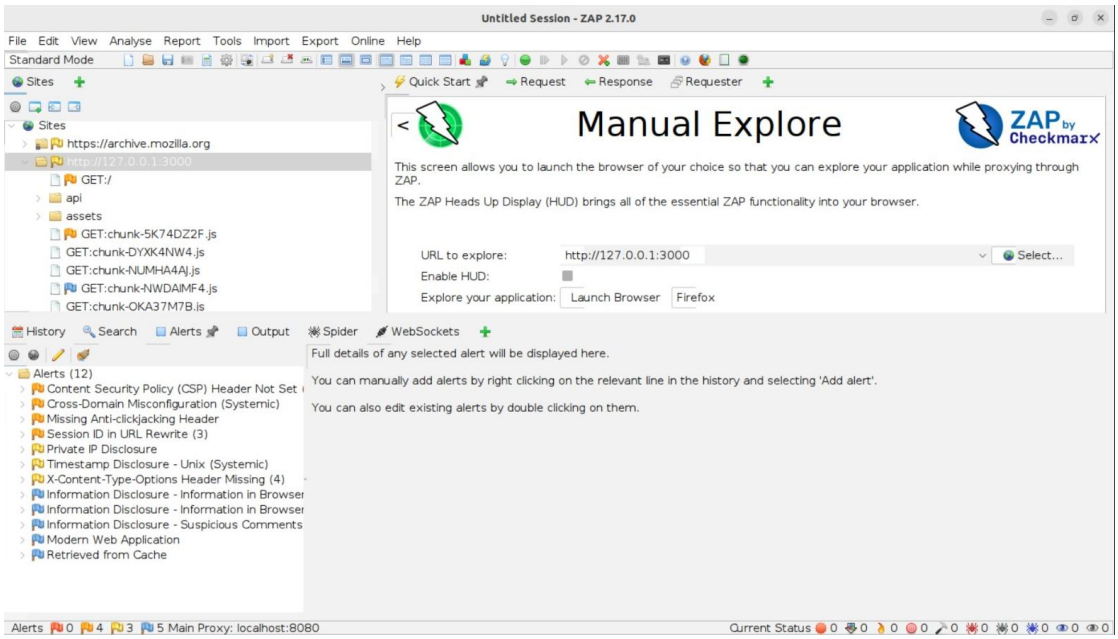




4. Methodology

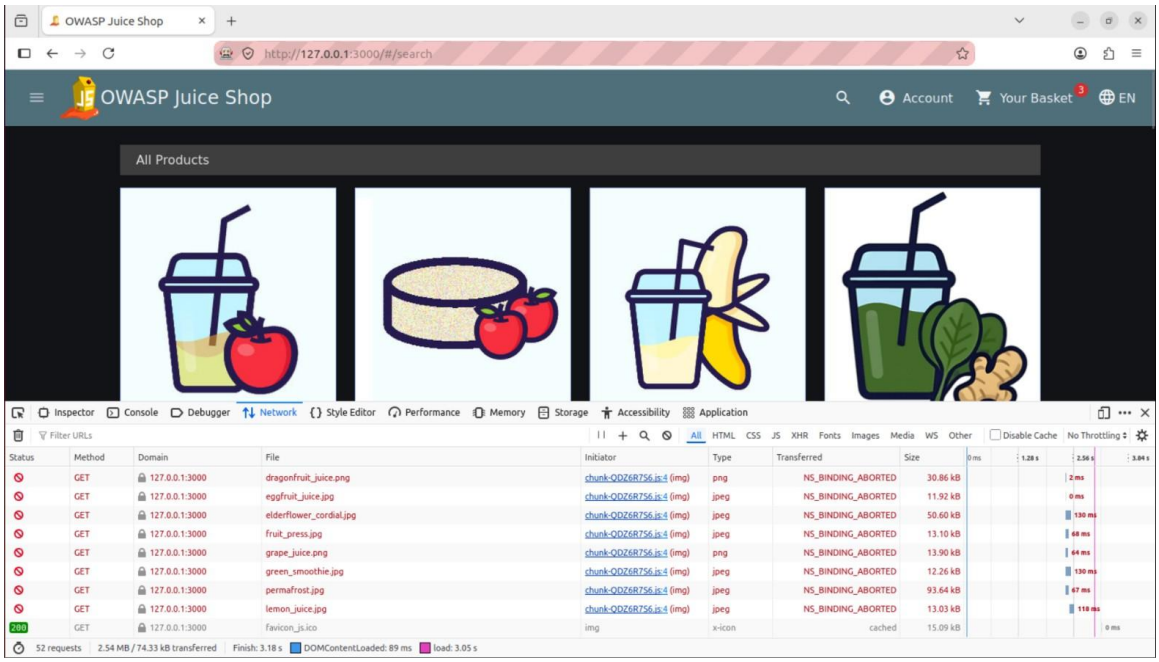
4.1 Manual Exploration

I opened OWASP ZAP and used the Manual Explore feature to connect ZAP with the local Juice Shop instance.



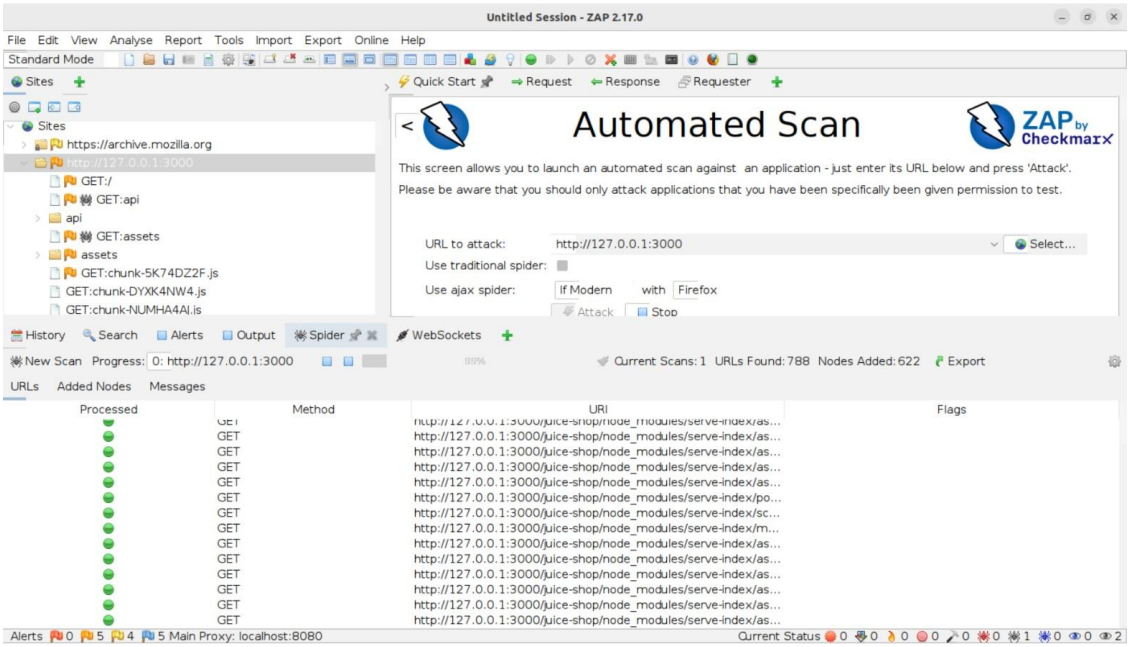
4.2 Browser Developer Tools

I used Firefox Developer Tools, especially the Network tab, to inspect requests, responses, and HTTP headers.



4.3 Automated ZAP Scan

I ran an automated scan in OWASP ZAP against http://127.0.0.1:3000. The scan discovered multiple alerts, including SQL Injection, Application Error Disclosure, and Content Security Policy Header Not Set.



5. Findings

Finding 1: SQL Injection

Category: Injection / SQL Injection

Risk Level: High

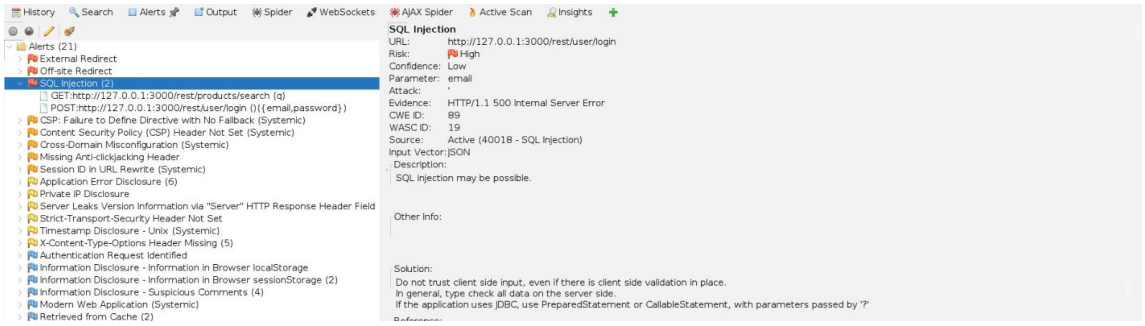
Affected Endpoint: http://127.0.0.1:3000/rest/user/login

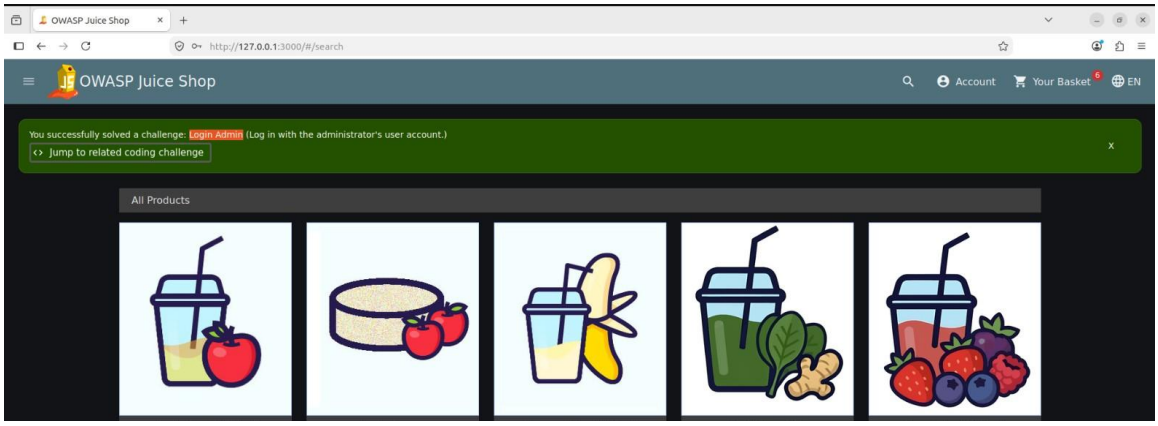
Description: OWASP ZAP detected a SQL Injection vulnerability in the login endpoint. I also manually verified this vulnerability by using an SQL injection payload in the login form. The login form accepted malicious SQL-style input and allowed authentication bypass.

Steps to Reproduce:

- Open OWASP Juice Shop at http://127.0.0.1:3000.
- Go to Account → Login.
- Enter the payload ' OR true-- in the email field.
- Submit the login form.
- The application logs in as the administrator account admin@juice-sh.op.
- Juice Shop displays the solved challenge message: Login Admin.

Evidence:





Impact: An attacker could bypass authentication and gain unauthorized access to user or administrator accounts. In a real-world application, this could lead to account takeover, unauthorized access to sensitive data, privilege escalation, modification or deletion of application data, and full compromise of application functionality.

Remediation: Use parameterized queries or prepared statements, never concatenate user input directly into SQL queries, validate and sanitize input, use generic login error messages, apply server-side input validation, and add security tests for authentication endpoints.

Finding 2: Application Error Disclosure

Category: Information Disclosure

Risk Level: Low / Medium

Affected Endpoints: http://127.0.0.1:3000/api and unexpected API paths such as http://127.0.0.1:3000/rest/user/login

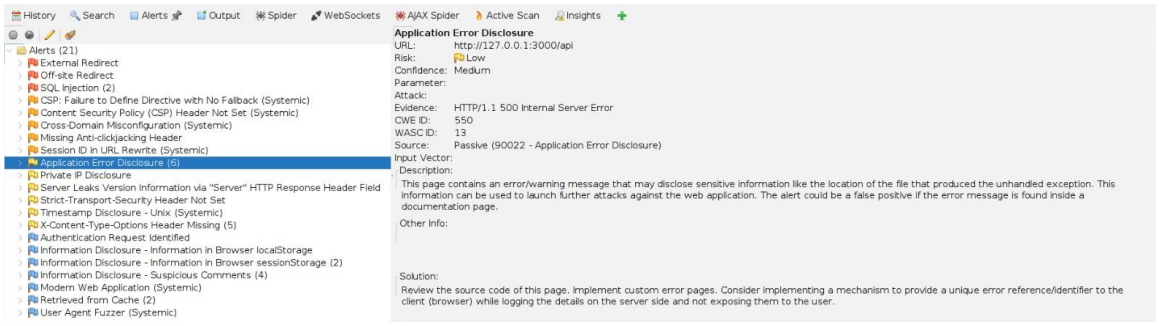
Description: The application disclosed detailed internal error information when unexpected or invalid paths were accessed. OWASP ZAP flagged this issue as Application Error Disclosure. Manual testing also confirmed that the application returned a detailed Express error page.

Example exposed information:
OWASP Juice Shop (Express ^4.22.1)
500 Error: Unexpected path: /rest/user/login
juice-shop/build/routes/angular.js
node_modules/express/lib/router
Layer.handle
trim_prefix

Steps to Reproduce:

- Open the browser.
- Visit an unexpected or invalid route such as http://127.0.0.1:3000/rest/user/login.
- Observe that the application returns a detailed 500 Error page.
- Review the error output and stack trace.

Evidence:



Impact: Detailed application error messages can help attackers understand the internal structure of the application. This information can be used to identify backend technologies, learn framework versions, discover internal file paths, understand route behavior, and support further attacks.

Remediation: Disable detailed error pages in production, return generic error messages to users, log full stack traces only on the server side, use centralized error handling middleware, avoid exposing framework versions and internal file paths, and configure production-safe error responses.

Finding 3: Content Security Policy Header Not Set
Category: Security Misconfiguration

Risk Level: Medium

Affected Endpoint: http://127.0.0.1:3000/

Description: The application did not include a Content-Security-Policy header in the HTTP response. OWASP ZAP detected this issue as Content Security Policy (CSP) Header Not Set. A CSP helps reduce the risk and impact of attacks such as Cross-Site Scripting by restricting which resources the browser is allowed to load.

Steps to Reproduce:

- Open OWASP Juice Shop at http://127.0.0.1:3000/.
- Open browser Developer Tools.
- Go to the Network tab.
- Reload the page.
- Select the main document request.
- Inspect the Response Headers.
- Confirm that the Content-Security-Policy header is missing.

Evidence:

Impact: Without a CSP, the browser has fewer restrictions on what scripts, styles, images, frames, and other resources can be loaded. If an XSS vulnerability exists, the missing CSP can increase the impact by making it easier for attackers to execute malicious scripts.

Remediation: Define and enforce a restrictive Content Security Policy. Start with Content-Security-Policy-Report-Only for testing, avoid unsafe directives such as unsafe-inline where possible, restrict resources to trusted sources, and monitor CSP violation reports before enforcing strict policy.

Example CSP header:
Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none';

6. Summary of Findings

#	Finding	Category	Risk
1	SQL Injection	Injection	High
2	Application Error Disclosure	Information Disclosure	Low/Medium
3	CSP Header Not Set	Security Misconfiguration	Medium

7. Conclusion

The audit confirmed that OWASP Juice Shop contains multiple intentionally vulnerable features. Using OWASP ZAP, browser Developer Tools, and manual testing, I identified and verified three security issues: SQL Injection, Application Error Disclosure, and Missing Content Security Policy Header.

These findings demonstrate the importance of secure coding practices, proper error handling, and strong HTTP security headers.